

# बोला मराठी (Bola Marathi) — Comprehensive End-to-End Development Guide

Welcome to the official technical documentation and development guide for **बोला मराठी** (Speak Marathi)! This guide is designed to help you understand, explain, and present the project architecture, features, and end-to-end technologies utilized.

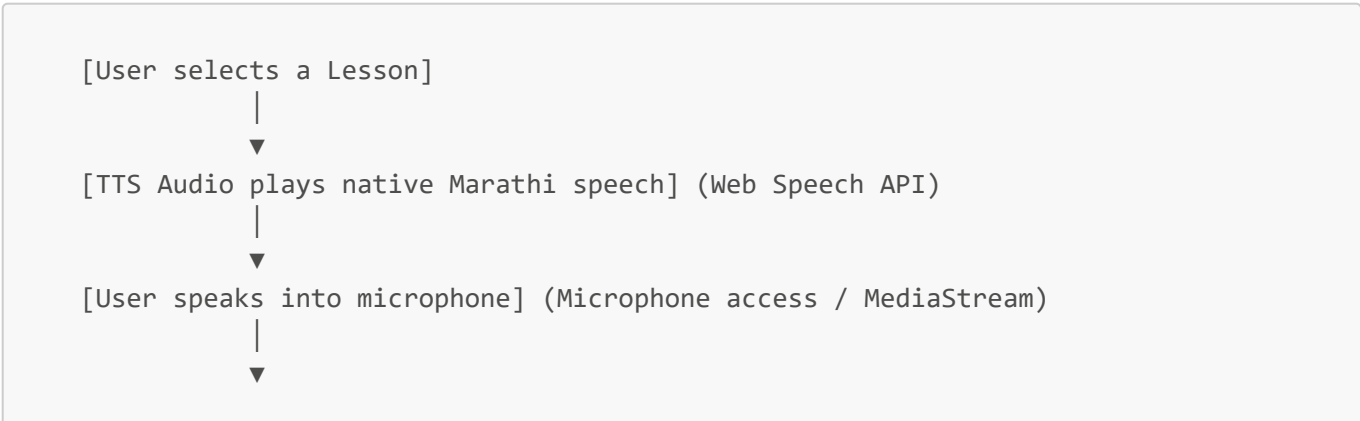
## 1. High-Level Architecture & Tech Stack

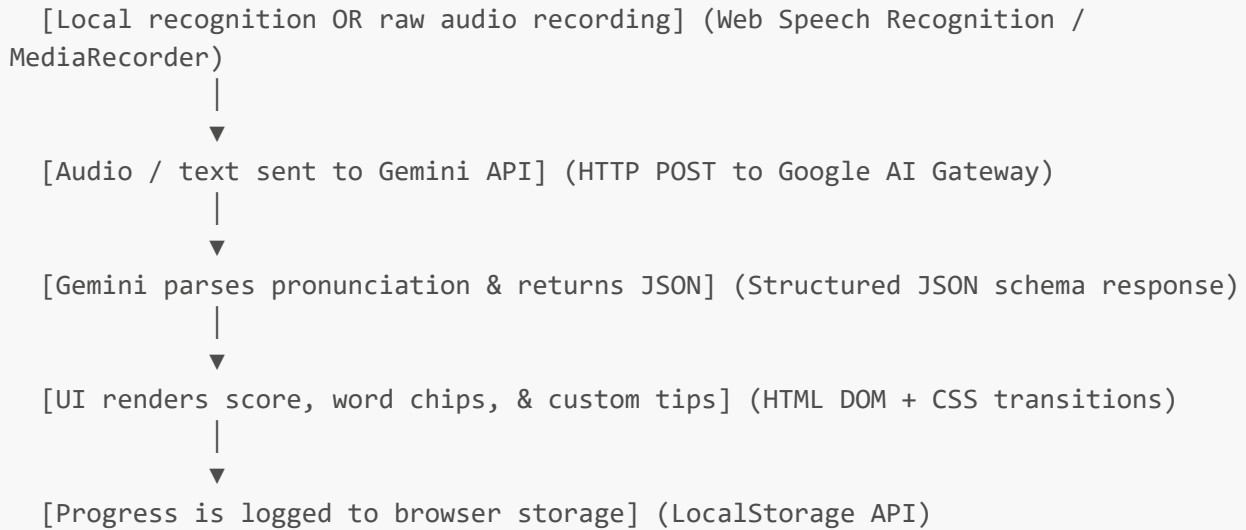
The application is built as a **Progressive Web App (PWA)** using a clean, modern, and zero-dependency Single Page Application (SPA) architecture. It requires no complex build tools (Webpack, Vite, etc.) or node dependencies, running entirely within the client's browser.

### A. Directory Structure

```
c:\Translation\Translation\  
├─ index.html           # Core SPA Entry Point & HTML5 Shell  
├─ manifest.json        # PWA Manifest (metadata, icons, colors)  
├─ sw.js                # Service Worker (offline-first caching logic)  
├─ css/  
│   └─ styles.css       # Design System (custom properties, animations,  
layouts)  
├─ data/  
│   └─ lessons.json     # Unified 12-Module, 100-Lesson, 835-Phrase Database  
├─ js/  
│   └─ app.js           # Application Controller (session timer, router)  
│   └─ ui.js            # View Engine (DOM renders, animations, event binds)  
│   └─ curriculum.js    # Curriculum Data Model & search helpers  
│   └─ audio.js         # TTS engine & Audio recording (Base64 audio export)  
│   └─ ai-feedback.js   # Gemini API Interface (Feedback assessment &  
translation)  
│       └─ progress.js  # LocalStorage stats tracker & Spaced Repetition  
engine
```

### B. End-to-End Flow of the AI Learning Loop





## 2. Core Modules & Technological Implementation

### A. The Front-End Shell & PWA Support

- **Responsive HTML5 SPA:** `index.html` defines the viewport and loads dependencies sequentially. The screen transition engine dynamically swaps view hierarchies in the `#app` container with smooth fade-in animations.
- **PWA Manifest (`manifest.json`):** Configured for standalone, app-like display (`display: standalone`) with a dark primary theme color (`#0a0a1a`). It defines icons, enabling native system installation on desktop systems (Chrome/Edge app) and mobile home screens.
- **Service Worker Caching (`sw.js`):** Employs an **Offline-First caching strategy** using the `CacheStorage` API. Static assets (CSS, JS, fonts, curriculum JSON) are cached locally under the name `bola-marathi-v4`. This allows the core lesson structure, video screens, translator views, and local speech synthesis to run without internet connectivity.

### B. Text-to-Speech (TTS) Engine (`js/audio.js`)

- **Web Speech API:** Uses `window.speechSynthesis` to convert written Marathi text to audio output dynamically.
- **Localization:** Programmatically queries the system voice list, prioritizing `mr-IN` (Marathi - India) voices.
- **Interactive Playback Speed:** Implements rate control so users can select:
  - **0.6x (Turtle Speed 🐢):** For detailed listening of complex vowels.
  - **1.0x (Normal Speed ▶):** Standard conversational speech.
  - **1.2x (Fast Speed ⚡):** Conversational speed practice.

### C. Speech-to-Text & Recording Engine (`js/audio.js`)

- **Native Recognition:** In Chrome/Android browsers, uses `webkitSpeechRecognition` configured for `mr-IN` to transcribe spoken Marathi directly in the browser.
- **Cross-Browser Audio Fallback:** For browsers lacking native Marathi speech recognition (like Safari/iOS), the engine records raw audio using the **MediaRecorder API** to capture a `.webm/.wav` blob. The file is converted to a base64 data string and dispatched to the Gemini API for cloud transcription.

- **Audio Visualizer:** Utilizing the **Web Audio API**, the app instantiates an **AudioContext**, attaches a **MediaStreamAudioSourceNode** from the microphone, and queries an **AnalyserNode** to draw a real-time pulsing audio wave.

#### D. Gemini AI Pronunciation & Translation Engines (**js/ai-feedback.js**)

- **Gemini API model (**gemini-1.5-flash**):** Chosen for low-latency responses, deep Marathi language comprehension, and robust support for Structured Output Schemas.
- **Speech Assessment Prompts:** Sends the expected Marathi phrase, user transcription/audio, and lesson metadata. It instructs Gemini to assess the speech and return a structured JSON response matching the following schema:

```
{
  "score": 88,
  "accuracy": "good",
  "feedback": "Your pronunciation of 'नमस्कार' was excellent, but try to
enunciate 'कसे' slightly more clearly.",
  "wordScores": [
    { "word": "नमस्कार", "score": 95, "transliteration": "namaskar" },
    { "word": "तुम्ही", "score": 85, "transliteration": "tumhi" },
    { "word": "कसे", "score": 75, "transliteration": "kase", "tip": "Stress
the 'ka' syllable." }
  ],
  "encouragement": "तुम्ही खूप छान शिकत आहात! 🌟"
}
```

- **AI Translator:** Employs the same model to perform bi-directional translation (English → Marathi, Marathi → English, Hindi → Marathi, Marathi → Hindi). It outputs the translated text along with a Latin phonetic transliteration guide to help users pronounce the output.

#### E. Gamified Progress Tracking (**js/progress.js**)

- **LocalStorage Persistence:** Stores user statistics locally in browser storage, removing the need for a backend database.
- **Game Elements:**
  - **Streak Tracker:** Computes consecutive daily study sessions.
  - **Experience Points (XP):** Earned dynamically based on pronunciation scores (e.g. +15 XP for scores ≥ 90).
  - **Spaced Repetition Review:** Flags phrases with scores below 70% as "review items," prompting users to practice difficult pronunciations.

---

### 3. Visual Layouts & Custom CSS Styling

- **Theme:** Premium glassmorphic dark mode (**background: #0a0a1a**) with linear gradients (Indigo/Violet theme).
- **Double Circular Charts (Dashboard):** Uses SVG vectors to render:
  1. **Daily Target Progress Ring:** Displays study minutes logged today relative to a 15-minute target.

2. **Course Completion Ring:** Computes overall lesson completion (`completed_lessons / 100`) and updates its stroke offsets dynamically. Both rings are placed side-by-side inside a responsive flexbox container (`.progress-rings-container`).

- **Weekly activity chart:** Renders a vertical bar chart dynamically using pure HTML/CSS flexbox. Bar heights are calculated relative to the maximum active minutes and show exact durations using hover tooltips.
- **Stats Grid:** A 2x2 grid displaying user statistics (Streak, XP, Total Lessons, Total Study Time).
- **Bottom Navigation Bar:** A glassmorphic navigation bar pinned to the bottom of the screen with smooth transitions, managing routing for **Lessons**, **Translate**, **Videos**, and **Dashboard**.

## 4. Video Lessons Library

- **12-Module Video Course:** Embeds educational YouTube videos mapped directly to each of the 12 curriculum modules.
- **Responsive Iframe Container:** Styled with a 16:9 CSS aspect ratio box (`padding-top: 56.25%`), ensuring videos adjust perfectly to any screen size without layout shifting.

## 5. Summary of Technologies Used End-to-End

| Layer           | Technology / API                                                        | Purpose                                                            |
|-----------------|-------------------------------------------------------------------------|--------------------------------------------------------------------|
| Frontend UI     | HTML5, Vanilla CSS3, JavaScript (ES6)                                   | Layout, styling, views, and routing                                |
| PWA Features    | Manifest V3, Service Worker API, CacheStorage                           | Desktop installation, offline capability, background asset caching |
| Audio Output    | Web Speech API ( <code>SpeechSynthesis</code> )                         | Native Marathi TTS text pronunciation                              |
| Audio Input     | Web Speech Recognition API ( <code>webkitSpeechRecognition</code> )     | Local translation of speech to Devanagari text                     |
| Voice Recording | MediaRecorder API, base64 encoder                                       | Capturing raw microphone audio for iOS/Safari fallbacks            |
| Audio Analysis  | Web Audio API ( <code>AudioContext</code> , <code>AnalyserNode</code> ) | Real-time waveform visualizer while recording                      |
| AI Assessment   | Google Gemini API ( <code>gemini-1.5-flash</code> )                     | AI speech assessment, grading, and structured JSON output          |
| AI Translation  | Google Gemini API ( <code>gemini-1.5-flash</code> )                     | Bi-directional English/Hindi/Marathi translation                   |
| Data Storage    | Web Storage API ( <code>localStorage</code> )                           | Saving user progress, XP, streaks, and settings offline            |
| Video Platform  | YouTube Player Embed API                                                | Streaming video tutorials for each module                          |

| Layer          | Technology / API                       | Purpose                                            |
|----------------|----------------------------------------|----------------------------------------------------|
| Hosting Server | Python <code>http.server</code> module | Lightweight local web hosting for desktop browsers |

## 6. How to Generate/Export this Guide as a PDF

You can easily generate a clean, styled PDF of this guide directly from your laptop:

### Method 1: Using VS Code (Easiest)

- 1. Open your project folder in **VS Code**.
- 2. Go to the Extensions view (`Ctrl + Shift + X`), search for **"Markdown PDF"** (by yzane), and install it.
- 3. Open `bola_marathi_guide.md` in the editor.
- 4. Press `Ctrl + Shift + P` (or `F1`), type `Markdown PDF: Export (pdf)`, and hit enter.
- 5. The PDF will save in your folder as `bola_marathi_guide.pdf`.

### Method 2: Using the Browser

- 1. In your command prompt, start your local Python server:

```
python -m http.server 8000
```

- 2. Open Chrome or Edge and navigate to `http://localhost:8000/bola_marathi_guide.md`.
- 3. Press `Ctrl + P` (Print).
- 4. Select **Save as PDF** as the destination and click **Save**.